Documentação Técnica – SGL

Oct 19, 2026 · @Ramon Vitor Dos Santos Silva

O SGL (Sistema de Gerenciamento de Logística) é um sistema desktop em C# Windows Forms com banco MySQL, que cadastra veículos, motoristas, rotas, preços de combustível e viagens, com login e consulta de viagens.

Tecnologias e estrutura

O sistema roda em .NET Framework 4.7.2/4.8 (Visual Studio) e acessa o MySQL pelo pacote NuGet MySql.Data 26.7.0, sempre com consultas parametrizadas.

Arquivo	Responsabilidade
Program.cs	Abre o FrmLogin; se o login for aceito, abre o Form1
Conexao.cs	Classe estática que cria a MySqlConnection (servidor, banco, usuário e senha)
Seguranca.cs	Gera e verifica o hash de senha (PBKDF2)
FrmLogin.cs	Tela de login
FrmCadastroUsuario.cs	Cadastro de usuários (ativo/inativo)
Form1.cs	Tela principal com 5 abas de CRUD (Veículo, Motorista, Rota, Preço de Combustível, Viagem)
Form1.Extras.cs	Grids (DataGridView), clique na linha, atualização após salvar e ViaCEP
FrmConsultarViagem.cs	Consulta de viagens com filtros de status e período e botão Detalhes

Padrão de nomes dos controles: `txt_` (caixa de texto), `btn_` (botão), `cmb_` (combo), `dtp_` (data), `dgv_` (grid), `chk_` (caixa de marcação) e `tab_` (aba).

Banco de dados

O banco `projeto_logistica` (MySQL) tem 6 tabelas: as 5 do PDF mais `usuario`. Uma viagem liga um veículo, um motorista e uma rota; cada um deles pode ter várias viagens (1:N). A tabela `preco_combustivel` não tem chave estrangeira.

Tabela	Campos
veiculo	veiculoID (PK, auto), modelo, placa (7, única), consumo_medio (Km/L), carga_maxima (Kg, opcional)
motorista	motoristaID (PK, auto), nome, cnh (11, única), telefone, cep, logradouro, bairro, cidade, uf
rota	rotaID (PK, auto), origem, destino, distancia (Km)
preco_combustivel	precoID (PK, auto), combustivel, preco (R$), data_consulta
viagem	viagemID (PK, auto), veiculoID (FK), motoristaID (FK), rotaID (FK), data_saida, data_chegada (opcional), situacao
usuario	usuarioID (PK, auto), nome, login (único), senha (hash), ativo (1 = ativo, 0 = inativo)

O script `banco.sql` cria o banco e as tabelas. Excluir um registro usado em uma viagem é bloqueado pela chave estrangeira (erro MySQL 1451), e placa, CNH ou login repetidos geram o erro 1062; o sistema mostra mensagens amigáveis para os dois casos. O AUTO_INCREMENT não reaproveita IDs excluídos, comportamento normal do MySQL.

Telas e regras de negócio

Cada aba do Form1 tem Salvar, Editar, Excluir e Consultar, mais um grid que lista os registros; clicar numa linha carrega o registro nos campos. Após salvar, os campos são limpos para o próximo cadastro.

Cadastro	Validações
Veículo	Placa com 7 caracteres; consumo médio maior que 0; placa única
Motorista	CNH com 11 dígitos; telefone com 10 ou 11 dígitos; CNH única; endereço via CEP
Rota	Origem e destino informados; a distância (Km) é preenchida automaticamente pela API ao sair dos campos

Cadastro	Validações
Preço de combustível	Preço menor que 100 (campo DECIMAL(4,2)); data da consulta
Viagem	Veículo, motorista e rota escolhidos em listas; data de chegada não pode ser anterior à saída

A situação da viagem é definida pela caixa "tem chegada": sem data de chegada = "Em Andamento"; com data de chegada = "Concluída".

A tela **Consultar Viagem** (FrmConsultarViagem) filtra por status (Todos, Em Andamento, Concluída) e por período da data de saída. Os filtros são opcionais e usam parâmetros SQL. O botão Detalhes mostra veículo, motorista, rota, datas e status da viagem selecionada.

Login, segurança e ViaCEP

O Program.cs abre o FrmLogin antes da tela principal. O login só é aceito se a senha confere com o hash salvo e o usuário está ativo (`ativo = 1`). Novos usuários são criados no FrmCadastroUsuario (nome, login, senha de no mínimo 4 caracteres, ativo/inativo); a senha fica oculta na digitação.

A senha nunca é gravada em texto: a classe Seguranca usa PBKDF2 (Rfc2898DeriveBytes, 10.000 iterações, salt aleatório) e salva `salt:hash` em base64 na coluna `senha`.

Na aba Motorista, o botão de buscar CEP consulta a API ViaCEP (`https://viacep.com.br/ws/{cep}/json/`) e preenche logradouro, bairro, cidade e UF. O endereço é gravado junto ao motorista ao salvar ou editar. Se não houver internet ou o CEP não existir, o sistema avisa.

Como instalar e executar

1. Instale o MySQL Server e o Visual Studio com o desenvolvimento para desktop (.NET Framework).
2. Execute o script `banco.sql` no MySQL para criar o banco `projeto_logistica` e as tabelas (inclusive `usuario` e as colunas de endereço do motorista).
3. Clone o repositório SGL no GitHub e abra a solução no Visual Studio.
4. Restaure os pacotes NuGet (MySql.Data 26.7.0) e confirme que o Form1.Extras.cs está incluído no projeto.
5. Em Conexao.cs, troque `SUA_SENHA` pela senha do seu MySQL. Não envie a senha real ao GitHub.

6. Compile, execute e cadastre o primeiro usuário pelo botão Cadastrar da tela de login.

Checklist do PDF e Oportunidades de Melhoria

Todos os itens do checklist do PDF estão implementados e testados; a única limitação é que o custo de combustível usa o preço mais recente, sem separar por tipo de combustível.

Tópico do checklist	Situação	Observações
1. Autenticação e Acesso	Sim	Login antes do sistema, com usuário e senha; cadastro de novo usuário; só entra quem está Ativo
2. Qualidade e Segurança do Login	Sim	Senha oculta; mensagens de erro claras; cadastro com nome, login e senha (guardada com PBKDF2)
3. Cadastro e CRUD básico	Sim	Criar, consultar, editar e excluir nas 5 abas; exclusão bloqueada se o registro está em uso (chave estrangeira)
4. Gerenciamento de Viagem	Sim	Viagem vincula veículo, motorista e rota; situação definida pela data de chegada (vazia = Em Andamento)
5. Consulta e Filtros	Sim	DataGridView com filtros de status e período; mostra placa, motorista, rota e status; botão Detalhes
6. Acompanhamento de Custos	Sim, com limitação	Preço registra tipo de combustível e data da consulta; custo estimado aparece em Detalhes, com o preço mais recente
7. Conexão e Estrutura de Dados	Sim	MySQL com as tabelas e chaves estrangeiras do SQL do PDF, mais `usuario` e as colunas de endereço
8. Qualidade Técnica	Parcial	Há tratamento de erros e avisos de campo em branco; o código fica nos formulários, sem camadas, e restam métodos vazios do designer
9. Funcionalidades Extras	Sim	Descritas na seção seguinte

Funcionalidades extras e diferenciais

- ViaCEP (opcional do PDF): endereço do motorista preenchido pelo CEP.
- Distância da rota automática: ao informar origem e destino, o km é calculado pelas APIs gratuitas Photon (localiza as cidades) e OSRM (distância por estrada). Exige internet; se falhar, o km pode ser digitado.
- Custo estimado da viagem: o botão **Custo da viagem**, na tela Consultar Viagem, calcula (distância ÷ consumo médio) × preço de combustível mais recente.
- Botão **Novo** em cada aba: limpa os campos e mostra o próximo ID do banco.
- Pesquisa em tempo real: uma caixa acima de cada grid filtra as linhas em todas as colunas enquanto o usuário digita.
- Campos de ID somente leitura: o número vem do banco, pelo botão Novo ou pelo clique numa linha do grid.
- Avisos de validação: o campo com erro recebe um ícone vermelho (`ErrorProvider`) e fundo rosado, e o aviso some quando o usuário corrige.
- Botão **Sair**: volta à tela de login sem fechar o sistema (o `Program.cs` repete o ciclo login → sistema). Fechar pelo X encerra o programa.
- Janela redimensionável: ao maximizar, abas e grids acompanham o tamanho da tela (propriedade `Anchor`); as telas de login e de cadastro de usuário também maximizam, com o conteúdo centralizado.
- Tema de cores automático (classe `Tema`): botões coloridos por função, grids com cabeçalho e linhas alternadas.
- Clique numa linha do grid carrega o registro nos campos; campos são limpos após salvar.
- Senhas guardadas com hash PBKDF2, nunca em texto.

Oportunidades de Melhoria

- Informar o tipo de combustível de cada veículo, para o custo usar o preço certo (hoje o banco não guarda essa relação).
- Tirar a senha do MySQL do código (arquivo de configuração fora do Git ou variável de ambiente). No GitHub o `Conexao.cs` usa `SUA_SENHA` .
- Derivar a situação da viagem diretamente da data de chegada, sem guardar a coluna `situacao` .
- Separar o código em camadas (repositórios), em vez de concentrar o acesso ao banco nos formulários.
- Trocar as APIs públicas de rota por uma com chave ou guardar a distância em cache, para não depender da disponibilidade delas.
- Perfis de acesso por usuário e relatórios de custo por período.